

Gbenga Dairo

4. Data Analysis and Writeup

- 1) Say the LocatorBot gives a move command u_t (N, S, E, W), the belief distribution over all possible Roomba locations evolves according to the law of total probability. Let $b_{t-1}(a)$ represents the prior belief that the Roomba is in cell a at time $t - 1$.

Thus, the predicted belief at time t for cell b is:

$$b_t^-(b) = \sum_{a \in A} P(x_t = b \mid x_{t-1} = a, u_t) b_{t-1}(a) \quad \text{equ(1)}$$

Where A represents the set of all open cells on the grid.

Hence, the transition model is defined below as:

$$\begin{aligned} P(x_t = b \mid x_{t-1} = a, u_t) = \\ 0.8, \text{ if there is motion and } b = \text{neighbor}(a, u_t) \text{ is open,} \\ 0.2, \text{ if move fails or blocked. So } b = a, \\ 0, \text{ otherwise} \end{aligned} \quad \text{equ(2)}$$

In cases where the intended move places the Roomba directly into the LocatorBot cell, equ (2) still suffice with the interpretation that the localization process ends (the system terminates successfully once $x_t = \text{LocatorBot}$). Otherwise, equ (1) redistributes the prior probability mass over all reachable cells

- 2) After moving, the LocatorBot activates its detector, which can yield a beep ($z_t = \text{beep}$) or no beep ($z_t = \neg \text{beep}$).

The posterior belief after receiving the sensor reading z_t is:

$$b_t(b) = \eta P(z_t \mid x_t = b) b_t^-(b) \quad \text{equ(3)}$$

Where η is a normalization constant ensuring that the posterior belief distribution sums to one. Furthermore, the likelihood that the detector beeps decays exponentially with the Manhattan distance $d(b, L)$ between candidate cell b and the LocatorBot's position L :

$$P(\text{beep} \mid x_t = b) = \exp[-\alpha(d(b, L) - 1)] \quad \text{equ(4)}$$

Where $\alpha > 0$ controls sensor sensitivity (small α = long range but noisy sensor, i.e., many false positives, and large α = short-range but low noise sensor, i.e., a few detections).

Thus, the full likelihood model is:

$$\begin{aligned} P(z_t | x_t = b) = \\ \exp[-\alpha(d(b, L) - 1)], \quad z_t = \text{beep}, \\ 1 - \exp[-\alpha(d(b, L) - 1)], \quad z_t = \neg \text{beep}. \end{aligned} \quad \text{equ(5)}$$

Note: Substituting equ (5) into equ (3) yields the normalized posterior $b_t(b)$, which becomes the new belief state for the next iteration.

- 3) The Developed Strategy improves upon baseline strategy 2 by embedding the above recursive Bayesian filtering process inside a decision-theoretic framework. Instead of greedily chasing the most probable cell, the strategy evaluates the expected utility of possible sensing or movement actions. At every time step, the agent performs the following:

a) Prediction (Motion model): Updates $b_t^-(b)$ via equ (1 – 2).

b) Correction (Sensor model): Refines $b_t(b)$ via equ (3 – 5).

c) Decision-Theoretic Update (Utility evaluation): The decomposable utility function is computed using:

$$U_t = -(w_m \times \text{moves} + w_s \times \text{senses} + w_u \times (1 - \text{belief certainty})) \quad \text{equ(6)}$$

where w_m , w_s and w_u are cost weights for motion, sensing, and residual uncertainty.

The developed strategy, therefore, reasons about the expected value of information, balancing exploration (reducing uncertainty) and exploitation (acting on confident beliefs). Also, unlike Baseline strategy 2, which constantly moves toward the MAP cell, the developed strategy uses equ (6) to select actions that maximize expected utility rather than immediate probability gain. To justify this approach theoretically, the developed strategy corresponds to a recursive Bayesian filter embedded within a decision-theoretic agent, which is consistent with axioms of utility theory, including:

a) Completeness and transitivity: where the agent can compare and prefer among possible actions based on their expected utilities.

b) Continuity and independence: where small changes in probabilities yield proportional changes in preference.

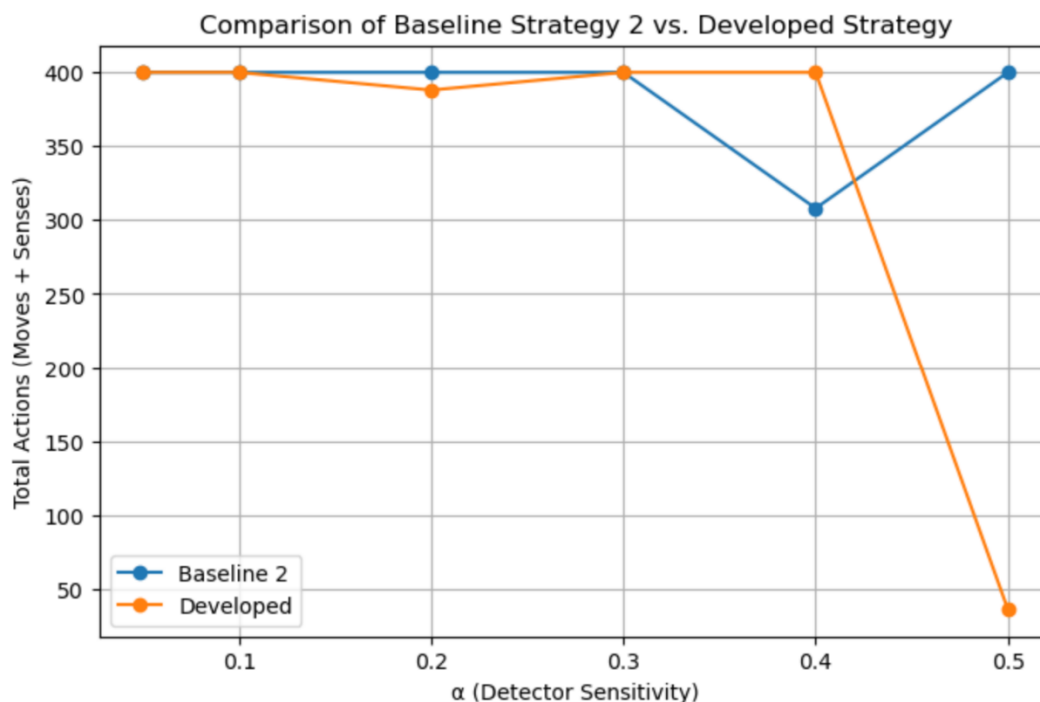
c) Monotonicity and decomposability: where higher belief certainty increases total utility in an additive way.

In brief, baseline strategy 2 employs probabilistic reasoning without the use of decision theory. In contrast, the developed strategy applies decision-theoretic reasoning over a

Bayesian belief state, thereby enhancing convergence efficiency and action rationality.

- 4) From the result (**Fig. 1; please check the Q4 report for code input and output**), we can deduce that at low α values, the sensor readings are nearly uninformative (too noisy), so both strategies require the maximum action cap (~ 400). However, at $\alpha = 0.5$, the developed strategy reduces the total number of actions (36 vs. 400) and achieves the highest normalized utility (0.514). This suggests that an increase in α causes the developed strategy to exploit more reliable sensor feedback and refines its belief faster, indicating that decision-theoretic reasoning becomes more effective as the sensor becomes informative. Additionally, we could infer that the developed strategy scales better with state-space complexity because recursive Bayesian filtering avoids the premature convergence that limits Baseline strategy 2.

Fig 1.



- 5) From the previous problem, the LocatorBot cannot move, and only Roomba executed moves. Here, generalizing the problem to allow the LocatorBot to also move employs the following formulas:
- a) Let x_t = Roomba's position at time t
 - b) Let L_t = LocatorBot's position at time t
 - c) Let $b_t(x_t)$ = belief distribution over Roomba's position
- Thus, the joint state of the system now becomes $s_t = (x_t, L_t)$.

Also, the agent can issue one of these two commands at each step:

1. Move command: This moves the LocatorBot to a new cell L_{t+1} .
2. Detect command: This runs the detector and updates the belief.

Here, the transition model from *equ* (1) becomes;

$$\mathbf{b}_t^-(\mathbf{x}_t) = \sum_{\mathbf{x}_{t-1}} \mathbf{P}(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{u}_t^{(R)}) \mathbf{b}_{t-1}(\mathbf{x}_{t-1}) \quad \text{equ(7)}$$

Also, the sensor model now depends on:

$$\mathbf{b}_t(\mathbf{x}_t) = \boldsymbol{\eta} \mathbf{P}(\mathbf{z}_t | \mathbf{x}_t, \mathbf{L}_t) \mathbf{b}_t^-(\mathbf{x}_t) \quad \text{equ(8)}$$

where $\mathbf{P}(\mathbf{z}_t | \mathbf{x}_t, \mathbf{L}_t)$ employ:

$$\mathbf{P}(\text{beep} | \mathbf{x}_t, \mathbf{L}_t) = \exp[-\alpha(\mathbf{d}(\mathbf{x}_t, \mathbf{L}_t) - 1)] \quad \text{equ(9)}$$

Note that the LocatorBot's own motion is deterministic:

$$\mathbf{L}_{t+1} = \mathbf{f}(\mathbf{L}_t, \mathbf{u}_t^{(L)}) \quad \text{equ(10)}$$

Where $\mathbf{u}_t^{(L)} \in \{\text{N, S, E, W, Stay}\}$.

Further, the decomposable utility function now includes the LocatorBot's motion cost:

$$\mathbf{U}_t = -(\mathbf{w}_m^{(R)} \mathbf{M}_t^{(R)} + \mathbf{w}_s \mathbf{S}_t + \mathbf{w}_m^{(L)} \mathbf{M}_t^{(L)} + \mathbf{w}_u [1 - \mathbf{C}_t]) \quad \text{equ(11)}$$

Where:

$\mathbf{M}_t^{(R)}$ = Roomba's total moves,

$\mathbf{M}_t^{(L)}$ = LocatorBot's total moves,

$\mathbf{S}_t$ = Total sensor activations,

$\mathbf{C}_t$ = current belief certainty,

$\mathbf{w}_m^{(R)}, \mathbf{w}_m^{(L)}, \mathbf{w}_s, \mathbf{w}_u$ = weighing factors.

Strategy Design: The following are the steps involved in executing the strategy:

Step1:

Here, before moving, compute the expected change in entropy (information gain) if the detector runs from the current location:

$$\Delta \mathbf{H}_{sense} = \mathbf{H}(\mathbf{b}_t^-) - \mathbf{E}_{z_t}[\mathbf{H}(\mathbf{b}_t | \mathbf{z}_t)] \quad \text{equ(12)}$$

Where $\mathbf{H}(b) = - \sum_x b(x) \log b(x)$ is Shannon entropy.

Step 2:

Then, estimate the potential improvement in expected certainty by relocating the LocatorBot closer to the most probable Roomba cell:

$$\Delta C_{move} = C_t(L'_t) - C_t(L_t) \quad equ(13)$$

For each reachable L'_t

Step 3:

Then, choose the action $a_t \in \{\text{Move}, \text{Sense}\}$ that maximizes expected utility gain:

$$a_t^* = \arg \max_{a_t} (E[U_t(a_t)] - \text{Cost}(a_t)) \quad equ(14)$$

Thus:

If $\Delta H_{sense} > \Delta C_{move}$, run the detector (sensing provides higher information gain).

If $\Delta C_{move} > \Delta H_{sense}$, move the LocatorBot toward the region of highest belief.

Theoretically, allowing the LocatorBot to move generally reduces the total number of actions, especially in large grids or when α is small (i.e., weak detector). Further, mobility improves localization because moving closer to the Roomba increases the effective signal-to-noise ratio of the sensor model, making beep/no-beep observations far more informative. Additionally, by repositioning itself, the LocatorBot reduces the spatial symmetry in the belief distribution and shrinks the region of cells that produce similar sensor readings, accelerating belief collapse. Also, in the utility-based framework, movement is not performed arbitrarily; the agent only moves when the expected reduction in uncertainty outweighs the movement cost. As a result, the LocatorBot tends to reposition itself strategically to maximize information gain, yielding faster localization than either Baseline Strategy 2 or the stationary developed strategy, as shown in the result output below (**Fig. 2; please check the Q4 report for code input and output**).

Fig 2

